

# Script oral de présentation

M1 Techniques d'Apprentissage Artificiel  
Université Paris 8 — 2024

---

## Régression Logistique & Arbre de Décision CART

Implémentés from scratch en C  
Prédiction du risque de crédit bancaire

---

Maxime Bronny — 19009314  
`maxime.bronny@etud.univ-paris8.fr`

Durée estimée : **9–10 minutes**  
(débit ~135 mots/min, pauses incluses)

## Contents

---

|                                              |    |
|----------------------------------------------|----|
| Introduction                                 | 2  |
| Slide 01 — Contexte & Problématique          | 3  |
| Slide 02 — Dataset & Enjeux                  | 4  |
| Slide 03 — Pipeline de prétraitement         | 5  |
| Slide 04 — Modèles : RL & CART               | 6  |
| Slide 05 — Résultats — Régression Logistique | 7  |
| Slide 06 — Résultats — Arbre de Décision     | 8  |
| Slide 07 — Comparaison RL vs CART            | 9  |
| Slide 08 — Seuil optimal & Performance C     | 10 |
| Slide 09 — Conclusion & Perspectives         | 11 |
| Outro                                        | 12 |
| Récapitulatif des timings                    | 13 |

## Introduction — Slide titre

---

**Objectif :** Accrocher l'auditoire, se présenter, annoncer le fil conducteur du projet en 15 secondes.

Bonjour à tous. Je suis Maxime Bronny, étudiant en M1 Techniques d'Apprentissage Artificiel à l'Université Paris 8.

Aujourd'hui je vais vous présenter mon projet de credit scoring : deux algorithmes de machine learning — une régression logistique et un arbre de décision CART — implémentés entièrement from scratch en langage C, sans aucune bibliothèque ML.

Le plan suit les neuf parties du diaporama, de la problématique jusqu'aux perspectives.

*Timing estimé :  $\approx 15$  s*

## Slide 01 — Contexte & Problématique

---

**Objectif :** Poser le contexte métier (enjeux financiers, réglementation), formuler la problématique centrale et annoncer l'angle original : une implémentation from scratch en C.

Commençons par le contexte. Les défauts de paiement engendrent chaque année des pertes se chiffrant en **milliards de dollars** selon le Fonds Monétaire International. Dans ce cadre, les banques sont soumises aux accords de **Bâle III**, qui les obligent à maintenir des modèles d'évaluation du risque robustes et explicables. Le credit scoring est donc un outil de décision critique : il conditionne directement l'accès au crédit de chaque emprunteur.

[*pause*] La **problématique centrale** de ce projet est la suivante : prédire si un emprunteur va faire défaut sur son prêt, à partir de **12 caractéristiques personnelles et financières**. Deux défis techniques s'y ajoutent : gérer un **déséquilibre de classes sévère** (81% de bons payeurs contre 19% de défauts), et implémenter les algorithmes **from scratch en C** — sans bibliothèques de machine learning.

L'objectif fil rouge est de comprendre finement les algorithmes en les codant soi-même, tout en démontrant que les performances sont proches de scikit-learn et que la vitesse est **×7 supérieure à Python**.

*[Transition] Maintenant que le cadre est posé, voyons le dataset sur lequel on travaille.*

*Timing estimé :  $\approx 60$  s*

## Slide 02 — Dataset & Enjeux

**Objectif :** Décrire le dataset (taille, variables, split), identifier les enjeux techniques liés aux données (déséquilibre, valeurs manquantes, encodage).

Le dataset provient de **Kaggle** et contient **32 581 emprunteurs**. On dispose de **12 variables** : 8 numériques et 4 catégorielles. La variable cible est `loan_status` : 0 pour pas de défaut, 1 pour défaut. Le split retenu est **80 % en entraînement et 20 % en test**.

[*pause*] Ce dataset pose trois enjeux techniques majeurs.

**Premier enjeu : le déséquilibre 81/19.** Avec une telle répartition, un modèle qui prédirait systématiquement “pas de défaut” obtiendrait déjà 81 % d’accuracy — ce qui est totalement trompeur. On devra donc s’appuyer sur le **F1-score** et l’**AUC-ROC**.

**Deuxième enjeu : les variables catégorielles.** Des variables comme `loan_grade` (grade de crédit de A à G) ou `cb_person_default_on_file` (historique de défaut Y/N) nécessitent un encodage spécifique intégré dès le chargement du CSV.

**Troisième enjeu : les valeurs manquantes.** `person_emp_length` est manquant à 2,75 % et `loan_int_rate` à 9,56 %. Ces valeurs sont imputées par la médiane, une stratégie adaptée au pattern aléatoire de ces absences. Enfin, la normalisation `StandardScaler` est calculée *uniquement* sur le train set pour éviter tout data leakage.

*[Transition] Voyons maintenant comment ces étapes s’enchaînent dans le pipeline de prétraitement.*

*Timing estimé :  $\approx 60$  s*

## Slide 03 — Pipeline de prétraitement

**Objectif :** Présenter les 5 étapes du pipeline en ordre, insister sur les choix techniques qui distinguent cette implémentation C (encodage intégré, Fisher-Yates, no leakage).

Le pipeline se décompose en **cinq étapes séquentielles**.

**Étape 1 : chargement CSV avec encodage catégoriel intégré.** Au lieu de lire le fichier puis d'encoder dans une deuxième passe, l'encodage se fait directement pendant le parsing. Cela réduit la complexité de  $O(2n)$  à  $O(n)$  — un gain non négligeable sur 32 000 lignes.

**Étape 2 : imputation des valeurs manquantes** par la médiane de chaque colonne. La médiane est préférée à la moyenne car elle est moins sensible aux valeurs aberrantes.

**Étape 3 : shuffle Fisher-Yates** avec un seed fixe `srand(42)`. Ce mélange garantit que les données ne sont pas biaisées par un ordre particulier dans le fichier source, et assure la **reproductibilité** des résultats.

**Étape 4 : split train/test 80/20** — soit **26 065 échantillons** en entraînement et **6 516 en test**.

**Étape 5 : normalisation StandardScaler**, formule Z-score  $x' = (x - \mu) / \sigma$ . La moyenne  $\mu$  et l'écart-type  $\sigma$  sont calculés *sur le train uniquement*, puis appliqués au test — ce qui évite toute contamination des données de test.

*[Transition] Le prétraitement est prêt. Décrivons maintenant les deux modèles implémentés.*

*Timing estimé :  $\approx 70$  s*

## Slide 04 — Modèles : RL & CART

**Objectif :** Présenter les hyperparamètres et la logique algorithmique de chaque modèle, en insistant sur leur complémentarité (linéaire vs non-linéaire).

Ce projet implémente deux modèles complémentaires.

**La Régression Logistique** repose sur la fonction sigmoïde  $\sigma(z) = 1/(1 + e^{-z})$  qui transforme une combinaison linéaire des features en probabilité de défaut. L'optimisation se fait par **gradient descent full batch** sur **1 000 itérations** avec un **learning rate**  $\alpha = 0,01$ . En pratique la convergence est atteinte autour de **600 itérations**. Le modèle apprend **12 paramètres** : 11 poids et 1 biais. La complexité totale est  $O(T \times n \times d) \approx$  **286 millions d'opérations** — ce qui reste très rapide en C.

[ *pause* ] **L'Arbre de Décision CART** suit l'approche de Breiman (1984) : partitionnement récursif binaire, avec le **critère de Gini** pour mesurer l'impureté des nœuds. Le pre-pruning est configuré avec **max\_depth = 7** et **min\_samples\_split = 20**. L'arbre résultant compte **77 nœuds** pour une profondeur réelle de 7.

L'idée clé : la régression logistique apprend une frontière *linéaire*, tandis que l'arbre peut capturer des *interactions non-linéaires complexes* entre variables. La comparaison de leurs résultats permettra de mesurer la séparabilité du problème.

*[Transition] Passons maintenant aux résultats de la régression logistique.*

*Timing estimé :  $\approx 70$  s*

## Slide 05 — Résultats — Régression Logistique

**Objectif :** Présenter et commenter les métriques de la RL en C, les comparer à sklearn, et interpréter les poids appris.

Sur l'ensemble de test, la régression logistique en C obtient : **80,11 % d'accuracy**, une précision de **48,66 %**, un recall de **52,06 %**, un F1-score de **50,31 %** et une **AUC-ROC de 80,28 %**.

La matrice de confusion montre **4 565 vrais négatifs**, **656 vrais positifs**, **692 faux positifs** et **604 faux négatifs**.

[ *pause* ] Ces chiffres révèlent une **bonne accuracy globale** mais une précision modeste sur la classe défaut. C'est la conséquence directe du déséquilibre 81/19 : sans pondération, le modèle favorise naturellement la classe majoritaire.

Concernant les **features les plus influentes**, le taux d'intérêt (`loan_int_rate`, poids  $|w| = 0,599$ ) arrive en tête — logique économiquement, un coût de remboursement élevé augmente le risque de défaut. Suivent le statut de propriétaire ( $|w| = 1,523$  pour `person_home_ownership`) et l'historique de défaut ( $|w| = 0,533$ ).

Enfin, en **comparaison avec sklearn**, notre implémentation C présente des écarts qui s'expliquent principalement par deux différences méthodologiques : sklearn utilise l'optimiseur L-BFGS et applique une régularisation L2 par défaut, ce que notre gradient descent simple ne fait pas encore.

*[Transition] Voyons maintenant si l'arbre de décision fait mieux.*

*Timing estimé :  $\approx 65$  s*



## Slide 06 — Résultats — Arbre de Décision

**Objectif :** Présenter les métriques de l'arbre, insister sur la précision exceptionnelle et valider la cohérence C vs sklearn (écart  $< 1\%$ ).

L'arbre de décision affiche des résultats nettement supérieurs : **93,23 % d'accuracy**, une précision de **96,48 %**, un recall de **67,46 %**, un F1-score de **79,40 %** et une **AUC-ROC de 91,59 %**.

La matrice de confusion est particulièrement frappante : l'arbre génère **seulement 31 faux positifs** sur 5 226 négatifs réels, soit un taux de 0,59 %. Dit autrement, quand l'arbre prédit un défaut, **il se trompe dans moins de 4 % des cas**. C'est une précision remarquable dans un contexte bancaire.

[ *pause* ] L'autre point important est la **validation contre sklearn** : l'écart est inférieur à 1 % sur toutes les métriques. Cela confirme que l'implémentation C est algorithmiquement correcte. Les 32 tests unitaires qui passent à 100 % vont dans le même sens.

*[Transition] Maintenant que nous avons les résultats des deux modèles séparément, comparons-les directement.*

*Timing estimé :  $\approx 55$  s*

## Slide 07 — Comparaison RL vs CART

**Objectif :** Montrer la supériorité de l'arbre sur toutes les métriques, quantifier l'impact métier (coûts FP/FN) et conclure sur la séparabilité du problème.

Le tableau comparatif est sans appel : l'arbre domine sur toutes les métriques. **+13,12 points d'accuracy**, **+47,82 points de précision**, **+29,09 points de F1-score** et **+11,31 points d'AUC-ROC**.

Sur les erreurs brutes, le constat est similaire : les faux positifs passent de **692 à 31** (95,5 %) et les faux négatifs de **604 à 410** (32,1 %).

[*pause*] Ce qui rend cette comparaison encore plus parlante, c'est l'angle métier. Dans le secteur bancaire, les deux types d'erreurs n'ont pas le même coût. Un faux négatif, c'est-à-dire accorder un crédit à quelqu'un qui fera défaut, coûte environ **4 168 dollars**. Un faux positif, refuser un bon payeur, représente une perte d'opportunité d'environ **1 056 dollars**. Le ratio est donc de **4 pour 1**.

En appliquant ces coûts, l'arbre de décision permettrait une économie de **environ 1,5 million de dollars** sur le jeu de test, soit une réduction de **43 % des coûts d'erreur** par rapport à la régression logistique.

Ce résultat confirme une hypothèse centrale : le problème de crédit scoring est **non linéairement séparable**, ce qui explique la supériorité nette de l'arbre.

*[Transition] Ces coûts asymétriques nous amènent naturellement à la question du seuil de décision et des performances computationnelles.*

*Timing estimé :  $\approx 70$  s*

## Slide 08 — Seuil optimal & Performance C

**Objectif :** Expliquer pourquoi le seuil par défaut de 0,5 n'est pas optimal, démontrer le gain de vitesse du C, et présenter les indicateurs de qualité logicielle.

Puisque les faux négatifs coûtent **4 fois plus** que les faux positifs, le seuil de décision par défaut à 0,5 n'est pas adapté. Une analyse sur différents seuils montre que **le seuil 0,3 est optimal** : il fait passer le recall de 52 % à **72,5 %** et réduit le coût total estimé de **3,25 millions à 2,83 millions de dollars**.

[*pause*] Sur le plan des **performances computationnelles**, le résultat est très net. Notre implémentation C traite le pipeline complet — chargement, prétraitement, entraînement et évaluation — en **0,433 seconde**, contre environ **3 secondes pour Python** avec scikit-learn. Cela représente un **speedup de  $\times 7$** . L'objectif initial était moins de 5 secondes : il est dépassé d'un **facteur 11,5**.

La qualité logicielle est aussi au rendez-vous : **32 tests unitaires** passent à 100 %, l'architecture est modulaire en cinq couches (*utils / data / preprocessing / models / evaluation*), le projet utilise **Docker pour la reproductibilité**, totalise **environ 1 200 lignes de C** et compile avec les flags stricts `-O2 -Wall -Wextra`.

*[Transition] Nous arrivons à la conclusion.*

*Timing estimé :  $\approx 70$  s*

## Slide 09 — Conclusion & Perspectives

**Objectif :** Récapituler les résultats clés, reconnaître honnêtement les limites, et projeter les améliorations futures.

Pour conclure, ce projet a atteint ses objectifs sur les quatre points principaux.

**Résultats :** La régression logistique atteint **80,11 % d'accuracy** et une **AUC de 80,28 %** — une bonne baseline. L'arbre de décision atteint **93,23 % d'accuracy** et une **AUC de 91,59 %** — c'est le modèle optimal pour ce problème. L'implémentation C est **7 fois plus rapide** que Python (0,43 s vs 3 s), et les **32 tests unitaires** valident la correction algorithmique vis-à-vis de sklearn avec un écart inférieur à 1 %. Enfin, ajuster le seuil de décision à 0,3 permet de réduire le coût métier de **3,25 millions à 2,83 millions de dollars**.

[ *pause* ] Ce projet a cependant plusieurs **limites identifiées** : absence de régularisation L1/L2, pas de rééquilibrage des classes (SMOTE ou class weights), encodage catégoriel figé sur ce dataset, et pas de cross-validation K-fold.

En termes de **perspectives**, à court terme il est prévu d'ajouter la régularisation L2 et les class weights. À moyen terme : implémenter une Random Forest en C et une K-fold CV. À long terme : paralléliser avec OpenMP et SIMD (AVX2), et exposer le modèle via une API REST C minimale.

Le message central reste celui du diaporama : implémenter from scratch, c'est comprendre finement les algorithmes — et comme on vient de le voir, c'est aussi atteindre des performances à la hauteur des bibliothèques standard, avec une vitesse d'exécution qui parle d'elle-même.

*Timing estimé :  $\approx 65$  s*

## Outro

---

Merci pour votre attention. Je suis disponible pour vos questions.

*Timing estimé :  $\approx 10$  s*

## Récapitulatif des timings

---

| Section                                  | Durée estimée       |
|------------------------------------------|---------------------|
| Introduction (slide titre)               | ≈ 15 s              |
| Slide 01 — Contexte & Problématique      | ≈ 60 s              |
| Slide 02 — Dataset & Enjeux              | ≈ 60 s              |
| Slide 03 — Pipeline de prétraitement     | ≈ 70 s              |
| Slide 04 — Modèles : RL & CART           | ≈ 70 s              |
| Slide 05 — Résultats RL                  | ≈ 65 s              |
| Slide 06 — Résultats CART                | ≈ 55 s              |
| Slide 07 — Comparaison RL vs CART        | ≈ 70 s              |
| Slide 08 — Seuil optimal & Performance C | ≈ 70 s              |
| Slide 09 — Conclusion & Perspectives     | ≈ 65 s              |
| Outro                                    | ≈ 10 s              |
| <b>Total estimé</b>                      | <b>≈ 9 min 30 s</b> |

*Note : estimation basée sur un débit oral d'environ 135 mots/minute, transitions et pauses incluses. Adapter selon le rythme réel lors des répétitions.*